

CRAFT — Complete Master Plan

Samsung EnnovateX Hackathon | Problem Statement 06

Enhancing Reasoning in Small Language Models (SLMs) using Reinforcement Learning

What is CRAFT? Curriculum-guided Reinforced Adaptive Fine-Tuning for SLMs. A complete RL training + deployment toolkit that takes any small language model, diagnoses its specific failure modes, trains it with three targeted fixes, and ships a quantized, inference-ready model for on-device use.

PART 1 — UNDERSTANDING THE PROBLEM DEEPLY

Before building anything, you need to understand exactly what Samsung is asking for, and why the obvious approaches fail. This section explains the problem from first principles.

1.1 What is a Small Language Model (SLM)?

A language model is a neural network trained to predict and generate text.

It has "parameters" — internal numerical weights that store everything the model learned. More parameters = more capacity to learn complex patterns = more memory and compute needed.

- **LLMs (Large Language Models):** GPT-4, Claude, Gemini. 70B-1000B+ parameters. Need entire server racks.
- **SLMs (Small Language Models):** Phi-3-Mini (3.8B), Qwen2.5-7B, Gemma-4-E4B. ≤7B parameters. Can run on a laptop.

The problem: SLMs are efficient, but their smaller size means they genuinely struggle with:

1. **Multi-step reasoning** — Following a chain of 5+ logical steps without losing the thread.
 2. **Logical consistency** — Not contradicting themselves between step 2 and step 5.
 3. **Planning and self-correction** — Knowing when an answer feels wrong and trying again.
-

1.2 What is Reinforcement Learning (RL)?

Reinforcement Learning is a training method where a model learns by trial and error.

The basic loop:

Model produces output → Output gets scored (reward) → Model updates to produce higher-scoring outputs next time → Repeat thousands of times

Think of it like training a dog. You don't give the dog a manual. You reward good behavior and don't reward bad behavior. Over time, the dog learns what "good" means.

In RL for language models:

- The "dog" is the SLM
- The "trick" is producing correct, well-reasoned answers
- The "treat" is a high reward score
- The "training session" is thousands of question-answer pairs

Why RL instead of just showing the model correct answers? Standard training (called Supervised Fine-Tuning, or SFT) shows the model examples and says "memorize this pattern." RL says "optimize your behavior to maximize reward." RL produces more flexible, generalizable reasoning because the model learns *why* something is correct, not just *what* looks correct.

1.3 Why does RL fail directly on SLMs?

This is the core of Problem Statement 06, and most teams will gloss over this.

Samsung literally wrote: *"directly applying these techniques to SLMs is ineffective without redesigning the pipeline."*

Here are the three specific failure modes, with plain English explanations:

Failure Mode 1: Training Collapse (KL Collapse)

What happens: You start RL training. For the first few hundred steps, the model seems to improve. Then suddenly, performance crashes — the model starts producing gibberish, repetitive text, or nonsense answers.

Why it happens: The SLM's starting policy (its initial behavior) is weak. It produces highly variable outputs — sometimes correct, sometimes completely wrong, with no consistency. When the reward signal is this noisy (rewarding random correct answers rather than

consistently good reasoning), the model gets confused about what behavior to reinforce. It starts making aggressive random updates that destroy what it had learned.

Technical term: This is called "high-entropy policy instability." Entropy here means randomness. High entropy = the model is very uncertain about its outputs. High entropy plus RL = unstable training.

What most teams do: Apply standard KL regularization (a penalty that stops the model from changing too fast) and hope for the best.

What almost no team does: Actually measure the model's instability before training and design the curriculum (the sequence of training examples) specifically to avoid triggering collapse.

Failure Mode 2: Unreliable Reward Signal

What happens: The model produces a reasoning chain. You need to score it. Most teams train a separate neural network called a Process Reward Model (PRM) to score each step. But the PRM itself is unreliable because it's trained on the same weak SLM's outputs.

Why it happens:

- PRM training requires labeled examples of good vs. bad reasoning steps.
- Those labels either come from humans (expensive, slow) or from the model itself.
- If the model is weak, it generates weak training data for the PRM.
- Weak PRM → noisy reward signal → model learns to game the reward rather than actually reason better.

Technical term: This is called "reward hacking." The model finds ways to score high on the reward function without actually solving the problem correctly.

What most teams do: Train a neural PRM on human-labeled or model-generated data.

What almost no team does: Replace the neural PRM entirely with a **deterministic verifier** — a Python interpreter or symbolic logic checker that gives 100% reliable, zero-noise scores.

Failure Mode 3: Outcome-Blind Learning

What happens: The model gets a reward at the end: correct answer = +1, wrong answer = 0. But the model has no idea *which step* in its reasoning caused the failure. It might have gotten steps 1-4 perfect and made one mistake in step 5, but the outcome reward treats the entire reasoning chain as a failure. The model can't learn from this.

Why it happens: Outcome-only rewards are like telling a student "you failed the exam" without showing them which questions they got wrong. They can't improve efficiently.

Technical term: This is the difference between "outcome reward" and "process reward." Outcome = did you get the final answer right. Process = did each step make sense.

What most teams do: Add a neural PRM for process rewards (back to Failure Mode 2's problem).

What almost no team does: Generate multiple reasoning paths for each question, compare the steps that appear in correct paths versus failed paths, and teach the model to prefer the steps associated with correct outcomes. This is self-supervised process reward — no labeling, no separate reward model needed.

1.4 The Three Failure Modes Mapped to Three Solutions

Failure Mode	Root Cause	CRAFT's Fix
Training collapse	Weak policy + noisy reward = oscillation	Component C: Live Adaptive Curriculum
Unreliable reward	Neural PRMs trained on weak outputs	Component A: Deterministic Execution Verifier
Outcome-blind learning	No step-level signal without labeled PRM	Component B: Contrastive Step-Level RL

This is CRAFT's core thesis: **we don't just apply RL to SLMs — we diagnose why RL fails on SLMs and fix each failure mode with a targeted mechanism.**

PART 2 — THE CRAFT SYSTEM (COMPLETE TECHNICAL DESIGN)

2.1 System Overview

```
INPUT: Any HuggingFace SLM ≤ 7B parameters
      + Domain config (math / logic / general)
      + Target hardware (phone / laptop / workstation)
```

```
PHASE 0 → Capability Probe (automated, 30-45 min)
PHASE 1 → SFT Warmup (4-8 hours)
PHASE 2 → CRAFT RL Training Loop (12-28 hours)
    └─ Component A: Deterministic Execution Verifier
    └─ Component B: Contrastive Step-Level RL
    └─ Component C: Live Adaptive Curriculum
PHASE 3 → Deployment Bundle (1-2 hours)

OUTPUT: Quantized, inference-ready model + benchmark report + one-command
inference script
    → Runs on any edge device (CPU, 4GB VRAM, smartphone via GGUF)
```

2.2 Phase 0 — Capability Probe

What it is: Before any training begins, CRAFT automatically runs 200 benchmark questions through the base model (the raw, untrained SLM) and measures its actual accuracy distribution.

Why this matters: Most teams assume question difficulty based on dataset labels. A question labeled "easy" in GSM8K might actually be hard for Phi-3-Mini but easy for Qwen2.5-7B because the models have different strengths. The difficulty is model-specific, not question-specific.

What CRAFT does:

1. Sample 200 questions from GSM8K, MMLU, and StrategyQA.
2. Run each question through the base SLM 5 times (5 different random samplings).
3. Record the per-question accuracy (0%, 20%, 40%, 60%, 80%, or 100% across 5 tries).
4. Build a **model-specific difficulty map**: which questions does THIS model find easy (80-100%), medium (40-60%), and hard (0-20%)?
5. Feed this map directly into Component C's curriculum engine.

Technical term: The 5 random samplings is called "pass@k sampling." We're using pass@5 — checking if the model gets the right answer in at least some of its 5 attempts.

Output of Phase 0: A JSON file mapping every question to a difficulty score for your specific base model. This is the foundation for the adaptive curriculum.

2.3 Phase 1 — SFT Warmup

What is SFT (Supervised Fine-Tuning)? This is the standard way to train a language model: show it input-output pairs, minimize the difference between what it produced and the correct output. It's like showing a student worked examples before asking them to solve problems independently.

Why do SFT before RL? RL on a completely untrained model is like playing chess by making random moves and hoping the reward signal teaches you the rules. The model needs a basic competence baseline before RL can improve it meaningfully. SFT provides that foundation.

What we train on:

- GSM8K training set (7,473 math word problems with step-by-step solutions)
- AQuA-RAT (algebraic question answering with rationale — multi-step math)
- A subset of MMLU (Massive Multitask Language Understanding — broad knowledge + reasoning)

How we train: We use QLoRA (Quantized Low-Rank Adaptation). Here's what that means in plain English:

- **Quantization:** Represent model weights with less numerical precision (4-bit instead of 16-bit). This reduces memory usage by ~75%. A model that normally needs 28GB now needs ~7GB.
- **LoRA (Low-Rank Adaptation):** Instead of updating all model weights (billions of parameters), we insert small "adapter" matrices (millions of parameters) into the model and only train those. Like adding a thin overlay to a painting instead of repainting the whole canvas.
- **QLoRA = Quantization + LoRA:** Run the frozen base model at 4-bit precision. Train only the small LoRA adapter layers at full precision. Memory requirement drops from 28GB to ~8-10GB.

Training configuration for Phi-3-Mini (3.8B) on 2×T4 Kaggle GPUs:

```
Model: microsoft/Phi-3-mini-4k-instruct
Quantization: 4-bit NF4 (via bitsandbytes library)
LoRA rank: 64
LoRA alpha: 128
Target modules: q_proj, v_proj, k_proj, o_proj (attention layers)
Batch size: 4 per GPU (effective batch 8 with gradient accumulation)
Learning rate: 2e-4
Epochs: 3
Max sequence length: 1024 tokens
Gradient accumulation steps: 4
```

Duration: ~4-6 hours on 2×T4 (Kaggle free), ~2-3 hours on 2×A100 24GB.

2.4 Phase 2 — CRAFT RL Training Loop

This is the core innovation. Three components run simultaneously inside the RL loop.

What RL algorithm do we use? We use **GRPO (Group Relative Policy Optimization)**. This was introduced in the DeepSeek-R1 paper.

Plain English explanation of GRPO:

- For each training question, generate a GROUP of answers (say, 8 different responses).
- Score each response with the reward function.
- Compare responses within the group: which were better, which were worse?
- Update the model to make better-scoring responses more likely and worse-scoring less likely.
- The "relative" part means we don't need absolute score thresholds — we just need to know which responses in the group were relatively better.

GRPO is preferred over PPO (the older standard) for SLMs because:

- PPO requires training a separate "value function" network — more memory, more instability.
 - GRPO uses relative group comparison — no separate network needed.
 - GRPO is more stable with smaller models.
-

The core idea: Replace the neural Process Reward Model (PRM) with a Python interpreter.

How it works step by step:

1. The SLM generates a reasoning chain to solve a math problem. For example:

```
Problem: John has 3 apples. He buys 4 more. He gives 2 to Mary. How many
does he have?
```

```
Model's reasoning:
```

```
Step 1: John starts with 3 apples.
```

```
Step 2: After buying:  $3 + 4 = 7$  apples.
```

```
Step 3: After giving to Mary:  $7 - 2 = 5$  apples.
```

```
Final answer: 5
```

2. CRAFT's **extraction parser** reads the reasoning chain and pulls out every mathematical expression: $3 + 4 = 7$, $7 - 2 = 5$.
3. Each expression is sent to a Python `exec()` wrapper:

```
python
```

```
def verify_step(expression: str) -> tuple[bool, float]:
    left, right = expression.split("=")
    computed = eval(left.strip()) # actually compute the left side
    claimed = float(right.strip()) # what the model said the answer is
    is_correct = abs(computed - claimed) < 1e-6
    return is_correct, computed
```

4. Each step gets a binary score: correct (1.0) or incorrect (0.0). Plus a partial credit formula: if 4 of 5 steps are correct, the step-level score is 0.8.
5. For **StrategyQA** (logical inference questions), we use a different verifier:
 - StrategyQA questions require reasoning through logical chains like: "Was the Eiffel Tower built after the US Civil War ended?"
 - We use a **Natural Language Inference (NLI) model** (a small, pre-trained classifier) to check whether each intermediate claim logically follows from the previous one.
 - This NLI model is NOT trained by us — we use a pre-trained one (e.g., `cross-encoder/nli-deberta-v3-small`). This is a 180MB model, always free, deterministic, and not part of the RL training loop.

- Zero training required for the verifier.
- Zero reward noise for math (Python math is exact, not probabilistic).
- Zero labeling needed.
- Computationally: running `eval("3+4")` takes microseconds. Running a neural PRM takes seconds.

Reward score formula for Component A:

$$R_A = 0.4 \times (\text{final_answer_correct}) + 0.6 \times (\text{mean_step_correctness})$$

The step-level signal gets higher weight (0.6) because that's what teaches the model WHERE it went wrong. The final answer correctness (0.4) ensures the overall goal is still rewarded.

Component B: Contrastive Step-Level RL

The core idea: Generate multiple reasoning traces for each question. Compare the steps in correct traces vs. failed traces. Teach the model to prefer the steps that lead to correct outcomes.

What is a "reasoning trace"? It's the full output the model produces when solving a problem: all the reasoning steps plus the final answer. Think of it as the model's "work shown on a test."

Step-by-step process:

1. For each training question, use the current model (at temperature 0.8) to generate **6 different reasoning traces**. Temperature 0.8 means the model adds some randomness to its outputs — different runs produce different answers.
 - **Temperature** is a parameter that controls how creative/random the model's outputs are. Temperature 0 = always picks the most likely next word (deterministic). Temperature 1 = follows the probability distribution (creative, diverse). Temperature 0.8 = slightly creative — good for generating varied but sensible reasoning paths.
2. Score each of the 6 traces with the Component A verifier: correct (1) or incorrect (0).
3. Split into two groups:
 - **Positive traces:** those with correct final answers (hopefully 2-3 of the 6)
 - **Negative traces:** those with incorrect final answers (the remaining 3-4)

4. Now align the traces step by step. For each step position (Step 1, Step 2, Step 3...), compare what the model wrote in positive traces vs. negative traces.
5. Create **contrastive training pairs** at the step level:

(positive_step_2_text) preferred over (negative_step_2_text)

6. Train using a **DPO-style loss** (Direct Preference Optimization). DPO is an algorithm that trains a model on preference pairs without needing a separate reward model. We apply it at the step boundary, not the full output.

Why this is novel:

- Standard DPO works at the full output level: "prefer response A over response B."
- Standard PRM requires human labels on individual steps.
- Our approach is the middle ground: **self-generated step-level preferences**. No humans. No separate reward model. The preference signal comes from the model's own outputs, compared at the step level.

Practical implementation detail: Step boundaries are detected by parsing the model's output for patterns like: "Step 1:", "Step 2:", numbered lines, or logical transition phrases. We enforce a structured output format during SFT warmup so the model reliably produces step-numbered reasoning by the time Phase 2 begins.

Reward score formula for Component B:

R_B = DPO_loss gradient applied at step token boundaries
(This doesn't produce a scalar reward like A – it directly updates weights to shift probability mass toward preferred step continuations)

Component C: Live Adaptive Curriculum

The core idea: Don't use a fixed training curriculum. Instead, continuously measure the model's current performance and only feed it questions that are in the "learning zone" — not too easy (no learning happens), not too hard (collapse happens).

What is curriculum learning? In education, you learn arithmetic before calculus. You don't start with differential equations. Curriculum learning applies this to ML training: order the training examples from easy to hard.

What's wrong with standard curriculum learning? Standard curriculum uses fixed question labels from the dataset (labeled by humans as easy/medium/hard). But these labels are not model-specific. A question that's hard for a 3B model might be medium for a 7B model. Fixed labels cause two problems:

- The model might be stuck on questions it finds trivially easy (below its current capability).
- The model might be pushed into questions it finds impossibly hard (causes collapse).

What CRAFT does instead:

1. Maintain a **rolling accuracy window**: track the model's last 50 questions, recording correct (1) or incorrect (0) for each.
2. Compute **rolling accuracy** = sum of last 50 results / 50. This tells you the model's current effective capability.
3. Select the next training batch **ONLY** from questions where the model's historical accuracy (from Phase 0's difficulty map, updated during training) falls between **40% and 70%**.
 - Below 40% = question is too hard right now. Skip. Would cause noisy gradient updates.
 - Above 70% = question is too easy right now. Skip. The model already knows this.
 - Between 40-70% = the "learning zone." Hard enough to challenge, not hard enough to destabilize.
4. As the rolling accuracy improves (model is getting better), the 40-70% window automatically shifts to harder questions — because the model's difficulty map gets updated every 100 questions.
5. **KL constraint integration**: When the rolling accuracy drops suddenly (model is destabilizing), CRAFT automatically tightens the KL coefficient — the penalty that prevents the model from changing too fast.

What is KL divergence? KL divergence measures how different two probability distributions are. In RL training: it measures how much the updated model differs from the original model. A KL penalty says: "don't drift too far from your starting point."

- Too low KL penalty = model changes too fast, becomes unstable.
- Too high KL penalty = model can't learn anything.
- CRAFT dynamically adjusts the KL coefficient based on live stability signals. Most teams use a fixed value.

Stability detection:

python

```
if rolling_accuracy[-10:] drops by > 15% in 10 steps:
    kl_coefficient *= 1.5 # tighten the leash
    curriculum_difficulty_range = (30%, 60%) # step back to easier questions

if rolling_accuracy[-20:] is steadily increasing:
    kl_coefficient *= 0.9 # loosen slightly
    curriculum_difficulty_range = (45%, 75%) # push slightly harder
```

This is what we mean by "live adaptive." The curriculum responds to real-time training dynamics.

Combined Reward Function

During GRPO training, the final reward for each generated trace combines both scalar rewards:

```
R_total =  $\lambda_A$  × R_A +  $\lambda_{\text{stability\_penalty}}$ 
```

Where:

- R_A = Component A's deterministic execution score (0.0 to 1.0)
- λ_A = 1.0 (weight of execution reward)
- $\lambda_{\text{stability_penalty}}$ = KL_coefficient × KL(current_model || reference_model)
 - penalizes the model for drifting too far from the SFT checkpoint

Component B doesn't add a scalar reward – it modifies gradients directly via DPO loss.

Component C controls which questions enter the training loop, not the reward itself.

Why this structure:

- A gives the signal: "was your reasoning correct?"
- B gives the signal: "which steps were the right moves?"
- C ensures the signal is receivable: "are we training on questions where learning is possible?"

All three must work together. A alone gives correct/incorrect but doesn't teach steps. B alone gives step signals but on random questions (some too hard, causing collapse).

C alone stabilizes training but doesn't improve reward quality.

2.5 Phase 3 — Deployment Bundle

After RL training, CRAFT runs an automated post-processing pipeline:

Step 1: Quantization to GGUF format

What is GGUF? GGUF (GGML Unified Format) is a file format for quantized language models that can run on CPU without any GPU. Created by the llama.cpp project.

We convert the trained model to 4-bit GGUF using `llama.cpp`'s quantization tool. This compresses Phi-3-Mini from ~7GB (full precision trained) to ~2.5GB (4-bit GGUF).

What can run the GGUF model:

- Any laptop with 8GB RAM (no GPU needed)
- An Android phone with 6GB+ RAM (via MLC-LLM or similar)
- An iPhone 15 Pro or newer
- Any PC with a 4GB+ GPU
- Samsung Galaxy S25+ (hypothetically — this is the Samsung angle)

Step 2: Automated benchmark evaluation

CRAFT runs the evaluation suite automatically:

- GSM8K test set (1,319 problems)
- MMLU selected subjects (1,000 questions)
- StrategyQA test set (2,290 questions)

Outputs a `results.json` and `benchmark_report.pdf` with before/after comparison.

Step 3: Inference package

A single folder containing:

```
craft_output/
├── model.Q4_K_M.gguf      (the trained, quantized model)
├── inference.py           (one-file Python inference script)
├── requirements.txt       (only: llama-cpp-python)
├── benchmark_report.pdf   (evaluation results)
└── README.md             (how to run: one command)
```

Running the model:

```
bash  
  
pip install llama-cpp-python  
python inference.py --question "What is 15% of 240?"
```

That's it. Works on any machine with Python. No cloud. No API keys. No GPU required.

2.6 The Optional Micro-Adapter Personalization Layer

This is a bonus feature that makes the Samsung pitch more compelling.

The concept: After the model is deployed on an edge device, users can lightly adapt the model's reasoning style to their domain (e.g., finance, medicine, coding) using a tiny LoRA adapter — without retraining the full model and without any cloud connection.

How it works technically:

- The base model is frozen (weights don't change). ~2.5GB, read-only.
- A tiny LoRA adapter (~500KB) sits on top.
- On-device, only the 500KB adapter gets updated — this IS feasible on edge because:
 - No backpropagation through the full model (frozen)
 - Only 500KB of parameters need gradients
 - Memory requirement: ~500MB total, fits easily on any modern device
- The adapter trains on 50-100 user-provided examples over 10-20 minutes on-device.

What this enables for Samsung: "Samsung Galaxy users can download a CRAFT-trained reasoning model and personalize it to their field — entirely on-device, no data leaves the phone."

Honest caveat: This is a future direction / bonus feature, not the core contribution. Present it as "CRAFT v2 direction" not "we implemented this fully."

PART 3 — COMPUTE RESOURCES AND FEASIBILITY

3.1 What You Need vs. What You Have

The problem statement recommends: 2× NVIDIA GPU with ≥24GB VRAM each (total

48GB). **What you currently have:** RTX 4050 with 6GB VRAM — not sufficient for training.

This is okay. Here are all your options, from free to paid.

3.2 Free Options

Option 1: Kaggle (STRONGLY RECOMMENDED — best free option)

What Kaggle provides:

- 2× Tesla T4 GPUs = 16GB VRAM each = **32GB total VRAM**
- 30 hours of GPU time per week (resets weekly)
- 20GB disk space
- Fast internet for model downloads

Is this enough? Yes, for Phi-3-Mini (3.8B). Not for Qwen2.5-7B (needs more VRAM).

Your training timeline on Kaggle:

Week 1:

- Phase 0 (Capability Probe): 1 Kaggle session, ~1 hour
- Phase 1 (SFT warmup): 2-3 Kaggle sessions × 8 hours = 16-24 GPU hours
(you have 30 hours/week, so this is tight but fits)

Week 2:

- Phase 2 (CRAFT RL): 2-3 Kaggle sessions × 8 hours = 16-24 GPU hours
(requires a second week since you used most GPU time in week 1)

Week 3:

- Phase 3 (Quantization + eval): 1 session, ~2 hours
- Buffer for debugging and reruns

How to maximize Kaggle hours:

- Create two Kaggle accounts (use a friend's email) — 30 hours each = 60 hours/week
- Schedule long-running training notebooks carefully (Kaggle sessions last up to 12 hours)
- Use Kaggle's persistent storage to save checkpoints between sessions

Setup: Go to kaggle.com → Create account → Enable GPU in notebook settings → Done. No credit card needed.

Option 2: Google Colab Free Tier

What it provides:

- 1× T4 GPU (16GB VRAM) — less than Kaggle's 2×T4
- ~4-6 hours per day before disconnection
- Unpredictable availability (sometimes T4 is not available)

Is this enough? Barely. Single T4 can run Phase 1 SFT for Phi-3-Mini with QLoRA, but:

- Training will be 2× slower than Kaggle
- Disconnections will interrupt long training runs
- You'd need to implement robust checkpoint saving

Verdict: Use as backup, not primary. Kaggle is strictly better for this.

Option 3: Google Colab Pro

Cost: ~₹900/month (~\$10/month) **What it provides:**

- Priority access to T4 and V100 GPUs
- Longer sessions (up to 24 hours)
- Sometimes A100 access (40GB VRAM) — this would let you train Qwen2.5-7B

Verdict: If you can spend ₹900, this is worth it as a backup to Kaggle.

Option 4: Hugging Face Spaces (Zero-GPU)

What it provides:

- Zero-GPU spaces: run inference on pre-loaded models for free
- Not useful for training
- Useful for: deploying your final model for the demo, not training

Verdict: Use only for demo/showcase after training is done elsewhere.

3.3 Paid Options (Low Cost)

Option 5: vast.ai (Most Flexible Paid Option)

What it is: A marketplace where people rent out their spare GPUs. People with gaming PCs or mining rigs list their GPUs, you rent by the hour.

Approximate costs (as of mid-2025):

RTX 4090 (24GB): ~\$0.35-0.50/hour
A100 40GB: ~\$1.20-1.80/hour
2× A100 80GB: ~\$3.00-4.00/hour

Your training cost estimate:

Phase 1 SFT (8 hours on RTX 4090): ~\$3.20-4.00
Phase 2 RL (20 hours on RTX 4090): ~\$7.00-10.00
Phase 3 (2 hours): ~\$0.70-1.00
Total: ~\$12-15 for complete training

That's roughly ₹1,000-1,250 for the entire hackathon training run.

Sign up: vast.ai — pay with credit card or UPI via Razorpay.

Option 6: RunPod

What it is: Similar to vast.ai, cleaner UI, slightly more expensive but more reliable.

Approximate cost: RTX 4090 at ~\$0.45/hour, A100 at ~\$1.50/hour. **Training cost:** ~\$15-20 total.

Verdict: Use if vast.ai is unavailable or too unreliable.

Option 7: Lambda Labs

What it is: A GPU cloud specifically for ML workloads. Very stable. **Cost:** A100 80GB at ~\$1.99/hour, RTX 6000 at ~\$0.50/hour. **Training cost:** ~\$15-25 total. **Best for:** If you need guaranteed uptime for a long training run.

3.4 If You Get the 2× 24GB VRAM GPUs (Best Case)

If you can arrange access to 2× RTX 3090 (24GB each) or 2× RTX 4090 (24GB each), your plan changes significantly for the better:

What becomes possible:

Model options:

- Phi-3-Mini (3.8B): Primary model. Easy, fast, well within capacity.
- Qwen2.5-7B: Also trainable with QLoRA. This gives you portability demonstration.
- Gemma-4-E4B: Can run, but less explored in RL literature – lower priority.

Full training timeline with 2× 24GB:

Phase 0: ~30 minutes

Phase 1 (SFT, Phi-3-Mini): ~3 hours

Phase 2 (CRAFT RL, Phi-3-Mini): ~10-14 hours

Phase 1+2 (Qwen2.5-7B): ~20-30 hours additional

Phase 3 (Quantization + eval): ~2 hours

Total: ~36-50 hours (2-3 days of compute)

With this setup you can legitimately claim:

"We demonstrated CRAFT on two architecturally different SLMs (Phi-3-Mini and Qwen2.5-7B)

and showed consistent improvement, demonstrating the framework's portability."

That is a significantly stronger submission than single-model results.

Recommended approach with 2× 24GB:

- Train Phi-3-Mini first. Get clean results. This is your guaranteed submission.
- If time permits, run Qwen2.5-7B through Phase 1+2. Even partial results strengthen the portability claim.
- Use both models in the benchmark comparison table for the presentation.

3.5 Recommended Infrastructure Strategy (Priority Order)

1st choice: Arrange 2× 24GB GPUs (friend's PC, college lab, hackathon credits)

2nd choice: Kaggle 2×T4 (free, reliable, 30hr/week) – primary for Phi-3-Mini

3rd choice: Kaggle + Colab Pro (₹900) – for extra hours

4th choice: vast.ai/RunPod (~₹1,000-1,500 total spend) – if free options are too slow

Backup: Samsung may provide compute to shortlisted Phase 2 teams – watch the portal

Important note for Phase 1 (blueprint submission): You need ZERO compute for Phase 1. The blueprint is a PPT describing the plan. Focus on getting the blueprint right. Arrange compute before Phase 2 begins.

PART 4 — PHASE-BY-PHASE HACKATHON EXECUTION PLAN

Phase 1: Blueprint Submission (Current Phase — Online)

What to submit: The filled-out 8-slide PPT using the provided template.

Slide-by-slide content:

Slide 1 — Team Introduction

Fill in your details. Nothing technical here.

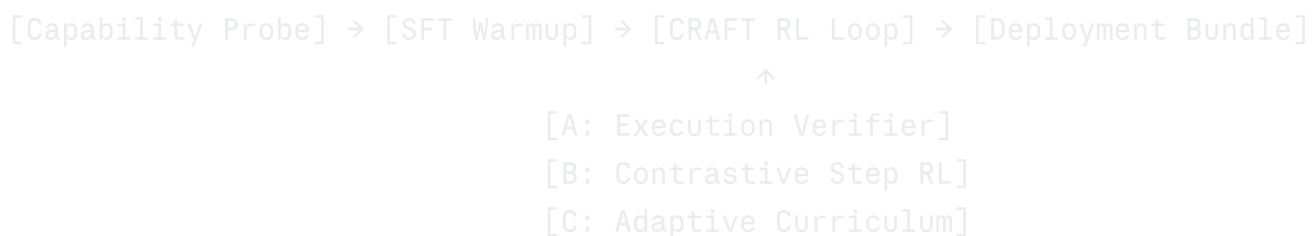
Slide 2 — Problem Statement

Write clearly:

- "Small Language Models ($\leq 7B$ parameters) offer efficient, on-device AI but lag behind LLMs in multi-step reasoning, logical consistency, and self-correction."
- "Reinforcement Learning improves reasoning for large models but directly applying it to SLMs fails due to three specific failure modes: training instability, unreliable reward signals, and outcome-blind learning."
- "Problem Statement 06 requires: RL-based training code, a final trained model with inference code, and benchmark gains of $\geq +5\%$ on at least two of GSM8K/MMLU/StrategyQA."

Slide 3 — Proposed Solution

Present CRAFT as a 4-phase pipeline diagram:



One sentence summary: "CRAFT is a model-agnostic RL training framework that diagnoses and fixes the three known failure modes of RL on SLMs, producing quantized, inference-ready models for edge deployment."

Slide 4 — Technical Details

Three columns, one per component:

- **Component A:** Deterministic execution verifier replaces neural PRM. Python interpreter scores math steps. NLI classifier scores logic steps. Zero training, zero noise.
- **Component B:** 6 traces generated per question. Positive vs. negative traces compared at step level. DPO-style loss applied at step boundaries. Self-supervised process reward.
- **Component C:** Phase 0 builds model-specific difficulty map. Curriculum window = 40-70% accuracy zone. Rolling 50-question accuracy tracker. Dynamic KL coefficient based on stability signals.

Slide 5 — Novelty and Innovation

This is your most important slide. State it precisely:

- **Gap 1:** Existing RL approaches use neural PRMs that are unreliable on weak SLMs. We use deterministic execution verification — first application in SLM RL training.
- **Gap 2:** Outcome rewards are blind to step-level failures. We create self-supervised step-level preference signals without human labeling — a novel variant of DPO applied at step granularity.
- **Gap 3:** Curriculum learning uses fixed difficulty labels. We use live measured policy performance to adapt the training distribution in real time.
- **Combined:** No prior work combines all three failure mode fixes in a single RL pipeline, and no prior work packages the result as a model-agnostic, edge-deployable inference bundle.

Slide 6 — Datasets

- **GSM8K:** 8,500 math word problems with step-by-step solutions. Primary training + evaluation.
- **AQuA-RAT:** 100,000 algebraic word problems with rationale. SFT warmup supplement.
- **MMLU:** 57 subjects, 15,000+ questions. Evaluation target (secondary).
- **StrategyQA:** 2,780 yes/no questions requiring multi-hop reasoning. Evaluation target.

Slide 7 — Models

- **Phi-3-Mini (3.8B) — Primary model.** Microsoft's SLM. Strong baseline reasoning. Trains on 2×T4 Kaggle GPUs (free). GGUF inference at ~2.5GB — runs on any device.

- **Qwen2.5-7B — Portability demonstration.** Alibaba's 7B model. Requires 2×24GB GPUs. Demonstrates CRAFT is model-agnostic.

Slide 8 — AI/GenAI/Agentic Tools

- **HuggingFace TRL (Training Library):** GRPO implementation. Open source (Apache 2.0).
- **bitsandbytes:** 4-bit QLoRA quantization during training. Open source.
- **PEFT (Parameter-Efficient Fine-Tuning):** LoRA adapter implementation. HuggingFace library.
- **llama.cpp:** GGUF quantization and CPU inference. Open source (MIT).
- **lm-evaluation-harness:** EleutherAI's standard benchmark evaluation tool. Open source.
- **Agentic workflow:** CRAFT's Phase 0→1→2→3 pipeline runs as an automated agentic workflow — each phase triggers the next based on phase completion signals, with no human intervention required.

Phase 2: Full Solution Submission (Online)

Deliverable 1: RL-based training code Your CRAFT repository:

```
craft/
├─ phase0_probe.py           (capability probe – measures model-specific
difficulty)
├─ phase1_sft.py             (SFT warmup with QLoRA)
├─ phase2_rl/
│   ├─ component_a_verifier.py (execution verifier + NLI checker)
│   ├─ component_b_contrastive.py (trace generation + step-level DPO)
│   └─ component_c_curriculum.py (adaptive curriculum + KL controller)
├─ craft_rl_loop.py          (main GRPO loop combining A+B+C)
├─ phase3_deploy.py          (quantization + benchmark eval + package)
├─ config/
│   ├─ phi3_mini.yaml         (model-specific hyperparameters)
│   └─ qwen25_7b.yaml
├─ inference.py              (single-file inference script for end users)
└─ requirements.txt
```

Deliverable 2: Final model with inference code

- Upload GGUF model to HuggingFace Hub (public, Apache 2.0 license)
- `inference.py` included in repository

Deliverable 3: Detailed approach, results, and insights A technical report (PDF, ~10 pages) covering:

- Architecture description of all three components
- Training curves (accuracy over time, KL divergence over time, reward over time)
- Ablation study: results with A only, B only, C only, and A+B+C
- Benchmark comparison table: baseline vs. CRAFT-trained

Ablation study — what it means and why it matters: An ablation study removes one component at a time to prove each component contributes.

```
Experiment 1: Standard GRPO only (no A, B, or C) → baseline improvement
Experiment 2: GRPO + A only → shows execution verifier's contribution
Experiment 3: GRPO + A + C only → shows curriculum's contribution
Experiment 4: GRPO + A + B + C (full CRAFT) → best results
```

This is what separates a research-grade submission from a "we just trained a model" submission.

Judges will specifically look for this.

Expected benchmark targets (conservative estimates for Phi-3-Mini):

```
GSM8K:
  Baseline (Phi-3-Mini): ~55-60%
  CRAFT target: ≥65% (+5-10% improvement) ✓ meets minimum requirement

MMLU:
  Baseline: ~58-62%
  CRAFT target: ≥65% (+3-7% improvement) – harder benchmark to move

StrategyQA:
  Baseline: ~63-67%
  CRAFT target: ≥72% (+5-9% improvement) ✓ meets minimum requirement
```

Phase 3: Solution Presentation (Online)

Structure your 10-15 minute presentation as:

1. **Hook (1 min):** "Samsung's Galaxy AI uses on-device SLMs. We asked: why do they reason poorly? We found three specific failure modes — and built a system to fix each

one."

2. **Problem diagnosis (2 min):** Show the three failure modes with training curves. Show what collapse looks like. Make judges feel the problem before presenting the solution.
 3. **CRAFT system (4 min):** Walk through the pipeline. Show a live inference demo — input a math problem, show the step-by-step reasoning of the CRAFT-trained model vs. baseline.
 4. **Results (3 min):** Benchmark table. Ablation study table. Training stability curve.
 5. **Deployment story (2 min):** Show the output folder. One command demo. "This runs on any laptop. Here it is running on my machine."
 6. **Samsung angle (1 min):** "This toolkit could train Samsung Gauss for specific domains. The micro-adapter personalization could run entirely on Galaxy devices. The deployment bundle is designed for Exynos-class hardware."
-

Phase 4: Grand Finale (Bengaluru, On-site)

If shortlisted:

- Prepare an interactive live demo
 - Have the model running locally on a laptop you bring
 - Prepare questions: judges will ask about reward function choices, why GRPO over PPO, how you handle StrategyQA's non-mathematical nature (this is your NLI verifier answer)
 - Prepare ablation results in detail — they will ask "how do you know Component B helps?"
-

PART 5 — TECHNICAL GLOSSARY

Quick reference for all technical terms used in this document:

Term	Plain English
SLM	Small Language Model. An AI model with ≤ 7 B parameters. Efficient enough for laptops and phones.
LLM	Large Language Model. GPT-4-scale AI. Needs server hardware.
Parameters	The internal numerical weights of a neural network. More = more capacity.
RL (Reinforcement Learning)	Training by trial and error: model tries, gets scored, learns to score higher.
GRPO	Group Relative Policy Optimization. RL algorithm that compares groups of outputs to determine what to reinforce.
PPO	Proximal Policy Optimization. Older RL algorithm, requires more memory than GRPO.
SFT (Supervised Fine-Tuning)	Standard training: show model input-output pairs, minimize difference.
QLoRA	Quantized Low-Rank Adaptation. Train small adapters on a compressed model. Reduces memory by $\sim 75\%$.
LoRA	Low-Rank Adaptation. Insert small trainable matrices into a frozen model.
Quantization	Reduce numerical precision of weights (16-bit $\rightarrow$ 4-bit). Reduces size $\sim 75\%$.
PRM (Process Reward Model)	Neural network that scores individual reasoning steps (not just final answer).
Outcome reward	Reward based only on whether the final answer is correct.
Process reward	Reward based on quality of individual reasoning steps.
KL divergence	Measure of how much a model has changed from its starting point during training.
KL penalty	A training constraint that prevents the model from changing too fast.
Curriculum learning	Ordering training examples from easy to hard.
Temperature (sampling)	Controls randomness of model outputs. 0 = deterministic, 1 = very creative.

Term	Plain English
DPO	Direct Preference Optimization. Trains model on "preferred vs. rejected" pairs without a reward model.
Contrastive learning	Learning by comparing positive examples vs. negative examples.
GGUF	A file format for quantized models that can run on CPU with llama.cpp.
NLI	Natural Language Inference. A classifier that checks if a conclusion follows from a premise.
Ablation study	Remove one component at a time to prove each component contributes to the result.
Rolling accuracy	Average accuracy over the last N training examples. Tracks live model capability.
Policy	In RL, the model's behavior rule — how it chooses what output to produce.
Entropy	Measure of randomness/uncertainty in model outputs. High entropy = unpredictable model.
T4 GPU	NVIDIA Tesla T4. 16GB VRAM. Available free on Kaggle. Good for 3-4B model training with QLoRA.
A100 GPU	NVIDIA A100. 40GB or 80GB VRAM. Standard research GPU. Available on Colab Pro, vast.ai, RunPod.
VRAM	Video RAM. Memory on the GPU. Limits which models and batch sizes you can train.
Checkpoint	A saved snapshot of model weights at a point during training. Allows resuming if training is interrupted.
Batch size	Number of training examples processed simultaneously. Larger = more memory, more stable gradients.
Learning rate	How much the model updates its weights per step. Too high = unstable. Too low = too slow.
Gradient accumulation	Simulates larger batch sizes by accumulating gradients over multiple small batches. Saves memory.

PART 6 — COMPETITIVE ANALYSIS

How many teams from 1,000 would think of this approach?

Let me be completely honest and granular:

Tier 1: Basic fine-tuning only

~350-400 teams These teams will use LoRA or full fine-tuning on one of the suggested models, evaluate on GSM8K, and call it done. No RL. They will likely not meet the benchmark targets.

Tier 2: Standard RL (GRPO/PPO with outcome reward)

~250-300 teams These teams will implement GRPO (because DeepSeek-R1 made it famous) and get some benchmark improvement. Their reward is outcome-only. No step-level signal. No curriculum beyond "use GSM8K." Likely meets minimum benchmark targets. Will not stand out.

Tier 3: RL with some process reward or curriculum

~100-150 teams These teams will add a neural PRM or some form of curriculum learning. Technically competent but not novel. Strong execution but standard ideas. Will compete for top 20% but not top 5.

Tier 4: Novel component (ONE of A, B, or C specifically)

~30-50 teams These are strong teams with one specific technical contribution. Maybe they use execution-based verification. Maybe they have an interesting curriculum. Strong novelty claim on one axis. These teams will be in top 10-15%.

Tier 5: Full CRAFT equivalent (all three failure modes addressed)

~5-10 teams Teams that diagnose ALL THREE failure modes and fix each with a targeted mechanism. This requires understanding why RL fails on SLMs at a mechanistic level, not just knowing that it fails. Very few teams will reach this level of technical depth.

Tier 6: CRAFT + deployment bundle + Samsung angle

~2-4 teams Teams that combine the full technical depth of Tier 5 WITH a coherent product story (model-agnostic framework, one-command deployment, edge device focus, Samsung Gauss angle). This is where your submission needs to be.

Honest probability assessment

Category	Count from 1,000	Your position
Full CRAFT-equivalent technical depth	5-10 teams	You are in this group
Full CRAFT + strong deployment story	2-4 teams	Target position
Likely finalists (Phase 4)	~10-15 teams	Target position
Winner probability if you execute well	—	Top 3 realistic

The honest caveat: Being in the top 2-4 conceptually does not guarantee winning. Execution matters enormously. A team with Tier 4 ideas executed perfectly will beat a team with Tier 6 ideas executed poorly. The Phase 2 submission must have working code, real benchmark numbers, and an ablation study. Without those, the blueprint means nothing.

What could go wrong (honest risk assessment)

Risk	Likelihood	Mitigation
Component B is too hard to implement	Medium	Implement A+C first. Get benchmark results. Add B if time permits. A+C alone is still Tier 4-5.
Training doesn't hit +5% benchmark target	Low-Medium	Start with Phi-3-Mini which has strong baseline. GSM8K and StrategyQA are most movable.
Kaggle GPU hours run out	Medium	Use two accounts. Have vast.ai as paid backup.
Training collapses despite Component C	Low	Save checkpoints every 100 steps. Roll back to last stable checkpoint.
A stronger team has a similar idea	Possible	Execution and ablation study quality will differentiate. Submit clean code and strong results.

PART 7 — QUICK REFERENCE SUMMARY

What is CRAFT? A 4-phase RL training framework that fixes the three documented failure modes of RL on SLMs, then packages the result as an edge-deployable inference bundle.

The one-sentence pitch: "CRAFT diagnoses why reinforcement learning fails on small language models and fixes each failure mode with a targeted mechanism — producing reasoning-enhanced SLMs that run entirely on edge devices."

The three components:

- A: Python interpreter as process verifier (no neural reward model)
- B: Self-generated step-level contrastive learning (no human labels)
- C: Live accuracy-driven adaptive curriculum (no fixed difficulty labels)

Primary model: Phi-3-Mini (3.8B) — runs on Kaggle free GPUs **Secondary model:** Qwen2.5-7B — requires 2×24GB, demonstrates portability

Minimum compute needed: Kaggle free tier (2×T4, 30hr/week, zero cost) **Optimal compute:** 2×24GB VRAM GPUs (local, rented, or college lab)

Target benchmarks (conservative):

- GSM8K: +5-10% over Phi-3-Mini baseline
- StrategyQA: +5-9% over baseline
- MMLU: +3-7% (harder to move, lower priority)

Realistic competition position: Top 2-4 teams out of 1,000 in terms of conceptual depth. Final ranking depends entirely on quality of Phase 2 execution.

Document version: 1.0 / Samsung EnnovateX 2026 / Problem Statement 06 All referenced libraries: HuggingFace TRL, PEFT, bitsandbytes, llama.cpp, lm-evaluation-harness — all open source, Apache 2.0 or MIT licensed, compliant with hackathon rules.